

บทที่ 3

เว็บลอจิกเว็บเซอร์วิส

3.1 คอนเวอร์เซชัน (Conversation)

สำหรับเว็บเซอร์วิส สามารถติดต่อกับหลายผู้ใช้ (Client) ในการทำงานหนึ่งการทำงานเว็บเซอร์วิส สามารถติดต่อแบบ อะซิงโครนัส (asynchronous) โดยใช้ คอนเวอร์เซชันในการรักษาสถานะของข้อมูลที่ติดต่อได้พร้อมกัน คอนเวอร์เซชัน มีปัญหาต้องจัดการ 2 อย่าง

1. รักษาสถานะ หรือข้อมูล โดยระบบการทำงานแบบ เว็บเซอร์วิส
2. จัดการ การเรียกใช้บริการ และ ตอบกลับไปยังเจ้าของ

3.1.1 การทำงานของคอนเวอร์เซชัน

คอนเวอร์เซชัน ทำงาน กับ คอนเทก (context) ที่ได้สร้างขึ้น โดย เว็บลอจิก เซิร์ฟเวอร์ (weblogic server) ในการสร้างคอนเวอร์เซชัน จะสร้างโดยไม่มีการซ้ำกัน เพื่อรักษา สถานการณ์ทำงานของ เว็บเซอร์วิส การสร้างคอนเทก เพื่อใช้ในการติดต่อผู้ใช้

3.1.1.1 การรักษาสถานะโดยใช้การซีเรียไลเซชัน (serialization)

สถานะหมายถึง เหตุการณ์ ที่เว็บเซอร์วิส ณ เวลาหนึ่ง ได้สร้างข้อมูล ที่มันเก็บไว้ ตัวอย่าง เซอร์วิสได้รับหมายเลขเสียภาษี จากผู้ใช้ และใช้หมายเลขนั้นในการหาข้อมูล เซอร์วิสนั้นเก็บหมายเลขผู้เสียภาษี เมื่อเวลาที่ต้องการใช้ หมายเลขผู้เสียภาษี ดังนั้น สามารถได้รับค่าหมายเลขผู้เสียภาษี เมื่อเวลาที่ต้องการใช้ หมายเลขผู้เสียภาษี เมื่อเวลาที่ต้องการใช้ หมายเลขผู้เสียภาษี เป็นตัวอย่างของข้อมูลที่มีสถานะมาเกี่ยวข้อง ลองพิจารณา คอมพิวเตอร์ที่มีการปิดบริการทันที ถ้าหมายเลขผู้เสียภาษี เก็บเป็นเพียงตัวแปรในบริการในบริการนั้น จะเกิดการสูญหายของค่านั้นถ้ามีการสูญหายของค่านั้นทันทีจากเหตุการณ์ ปิดคอมพิวเตอร์

3.1.1.2 จัดการข้อมูลผ่านกระบวนแบบไม่ซ้ำ

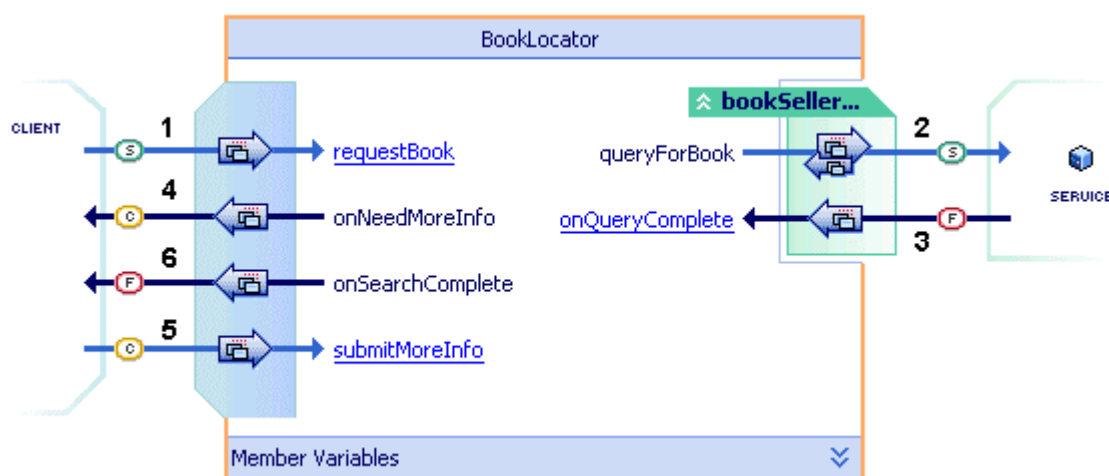
คอนเทกถูกสร้างเพื่อใช้บริการ โดยใช้ในการติดต่อกัน รวมถึงข้อมูลที่เก็บในตัวแปร รวมไปถึงการระบุผู้ใช้ในการใช้บริการตั้งแต่เริ่มเรียกใช้ข้อมูล เว็บเซอร์วิสอาจมีการเรียกใช้การทำงานมากมาย ณ เวลาเดียวกัน จากผู้ใช้คนเดียวหรือหลายคน ในการติดต่อ เซอร์วิสต้องการ จัดจำสถานะเพื่อใช้ในการจัดการการร้องขอและการตอบกลับอย่างมีประสิทธิภาพ

ตัวอย่าง

- แต่ละคอนเวอร์เซชัน เริ่มการทำงานโดย เว็บล็อกเชิร์ฟเวอร์ สร้าง หมายเลขการติดต่อเฉพาะที่ใช้เรียก คอนเวอร์เซชัน หมายเลขคอนเวอร์เซชันจะสัมพันธ์กับผู้ใช้ (เจ้าของ การเรียกใช้)
- แต่ละการทำงานจะมีผลกับข้อมูลที่ถูกเรียกโดยผู้ใช้ หมายเลขคอนเวอร์เซชัน ในการระบุ เจ้าของการเรียกใช้ เพราะฉะนั้น ผู้ใช้จะใช้หมายเลขคอนเวอร์เซชันในการติดต่อ

3.1.2 วงจรชีวิตของคอนเวอร์เซชัน

คอนเวอร์เซชันมีวงจรชีวิตของมันเอง จะเริ่มต้นเมื่อผู้ใช้เรียกบริการทางเว็บโดยใช้เมธอด “เริ่ม” (start) และทำงานโดยใช้เมธอด “ต่อไป”(continue) และจบโดยใช้เมธอด “จบ” (finish) ในการเรียกใช้ เมธอดครั้งแรกจะมีการสร้างคอนเทค เพื่อใช้ในการติดต่อ และการเรียกใช้อื่นอื่น มีการทำซีเรียวลีเซชัน เพื่อแก้ไขข้อมูลและเมื่อจบการทำงานของ คอนเวอร์เซชัน เว็บล็อกจะมีการคืนทรัพยากรและทำลาย สถานะที่จัดการโดยเว็บเซอร์วิส



รูปที่ 3-1 วงจรชีวิตของคอนเวอร์เซชัน

3.2 ทรานแซกชัน

3.2.1 เหตุผลที่ต้องใช้ทรานแซกชัน

ตัวอย่างปัญหาที่ต้องใช้ทรานแซกชันในการแก้

■ Atomic Operation

กรณีที่ต้องการทำคำสั่งหลาย ๆ คำสั่ง โดยมองทั้งหมดเป็นเหมือนคำสั่งเดียว (atomic operation) เช่น กรณีบัญชีธนาคาร เมื่อต้องการโอนเงินจากบัญชีหนึ่งไปยังอีกบัญชีหนึ่ง ต้องถอนเงินออกจากบัญชีหนึ่ง จากนั้นจึงฝากเงินเข้าอีกบัญชีหนึ่ง ถ้ามีคำสั่งใดคำสั่งหนึ่งล้มเหลว ต้องให้คำสั่งทั้งคู่ล้มเหลว ไม่เช่นนั้น

นั่นยอดเงินคงเหลือในบัญชีจะไม่ถูกต้อง จึงต้องไม่เกิดสถานการณ์ที่คำสั่งหนึ่งทำงานสำเร็จ ในขณะที่อีกคำสั่งทำงานล้มเหลว เนื่องจากทั้งสองคำสั่งนั้นเป็นทรานแซกชันเดียวกัน

การแก้ปัญหานี้วิธีหนึ่งคือการใช้ฟังก์ชันในการดักจับข้อผิดพลาด (exception) ดังตัวอย่างต่อไปนี้

```
try {
    // Withdraw funds from account 1
}
catch (Exception e) {
    // If an error occurred, do not proceed.
    return;
}

try {
    // Otherwise, deposit funds into account 2
}
catch (Exception e) {
    // If an error occurred, do not proceed,
    // and redeposit the funds back into account 1.
    return;
}
```

ตัวอย่างโค้ดข้างต้น ถอนเงินจากบัญชีที่ 1 ก่อน หากเกิดข้อผิดพลาดขึ้น แอปพลิเคชันจะจบการทำงาน และไม่มีผลของการทำงานใด ๆ เกิดขึ้น หากไม่เกิดข้อผิดพลาดก็จะฝากเงินเข้าไปยังบัญชีที่ 2 ในขณะที่ฝากเงินนั้น หากเกิดข้อผิดพลาด จะต้องทำการฝากเงินกลับเข้าไปในบัญชีที่ 1 ใหม่ จากนั้นจึงจบการทำงาน

ปัญหาที่เกิดขึ้นจากการแก้ปัญหาดังวิธีนี้คือ

- โปรแกรมมีความซับซ้อน
- ต้องตรวจสอบทุกความเป็นไปได้ที่จะทำให้การทำงานผิดพลาดในทุก ๆ ขั้นตอน และต้องเขียนโค้ดเพื่อยกเลิกการเปลี่ยนแปลงที่ได้ทำไปก่อนที่จะเกิดความผิดพลาดนั่นเอง
- ในการตรวจสอบความผิดพลาด แทบจะไม่สามารถควบคุมได้หากกระบวนการทั้งหมดมีความซับซ้อน เช่นการปรับปรุงข้อมูลทางการเงิน ที่ต้องอาศัยการทำงานทั้งหมด 10 ขั้นตอนด้วยกัน จะต้องหาวิธีการในการรองรับการทำงานที่ผิดพลาดในทุกขั้นตอนที่เป็นไปได้

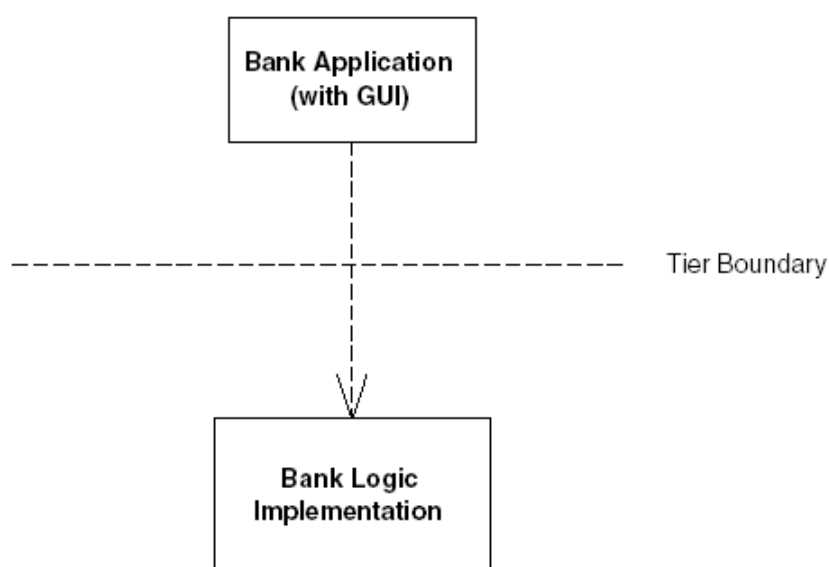
- การตรวจสอบการทำงานได้ยาก เพราะต้องทำการจำลองการทำงานที่ผิดพลาดที่เกิดขึ้นได้ในทุกขั้นตอน

ในทางอุดมคตินั้นต้องการวิธีการที่สามารถทำคำสั่งทั้งสองได้ โดยมองเป็นคำสั่งใหญ่เพียงคำสั่งเดียว และรับประกันว่าคำสั่งทั้งสองจะต้องทำงานสำเร็จทั้งคู่ หรือทำงานผิดพลาดทั้งคู่เท่านั้น

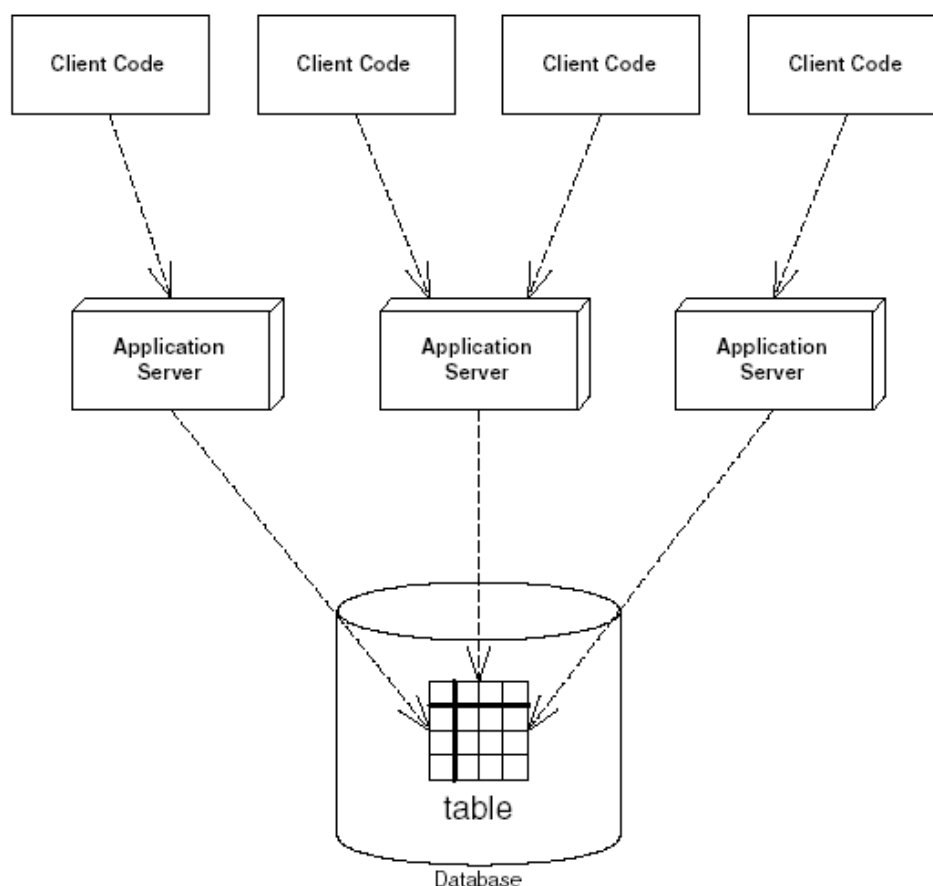
- การทำงานผิดพลาดของเครื่องคอมพิวเตอร์ หรือระบบเครือข่าย

จากตัวอย่างข้างต้น ถ้าลอจิกต่าง ๆ ที่ทำงานกับบัญชีธนาคารถูกกระจายบนเครือข่ายโดยดีพลอยแบบมัลติเทียร์ เพื่อเหตุผลทางด้านความปลอดภัย , การเพิ่มขยาย หรือประโยชน์จากการแบ่งโปรแกรมออกเป็นโมดูล ในการดีพลอยแบบมัลติเทียร์ โคลเอนต์ที่ต้องการใช้งานแอปพลิเคชันบัญชีธนาคารนี้ จะต้องกระทำผ่านทาง remote method invocation แสดงในรูปที่ 3-2 ทำให้ต้องคำนึงถึงเรื่องของการทำงานผิดพลาดและความน่าเชื่อถือ เช่น ระบบเครือข่ายแลกรระหว่างที่โปรแกรมบัญชีธนาคารกำลังทำคำสั่งใด ๆ อยู่ แม้ว่าเมื่อระบบเครือข่ายแลกรจะมีการสร้าง exception ขึ้นมา แต่การแลกรของระบบเครือข่ายนั้นอาจจะเกิดขึ้นก่อนหน้าการถอนเงิน หรืออาจเกิดขึ้นภายหลังการถอนเงินก็ได้ ซึ่งไม่อาจแยกความแตกต่างระหว่าง exception ที่เกิดขึ้นจากกรณีทั้งสองนี้ได้เลย

ในความเป็นจริง ระบบเครือข่ายไม่ได้เป็นจุดเดียวที่ทำให้เกิดปัญหานี้ขึ้น เนื่องจากข้อมูลบัญชีธนาคารก็คือข้อมูลแบบ persistent ที่อยู่ในฐานข้อมูล มีความเป็นไปได้เสมอที่ระบบฐานข้อมูลจะแลกรหรือเครื่องคอมพิวเตอร์ที่ดีพลอยระบบฐานข้อมูลไว้จะเกิดแลกร ถ้าหากการแลกรนั้นเกิดขึ้นในระหว่างที่กำลังเขียนข้อมูลลงไปในฐานข้อมูล อาจทำให้ฐานข้อมูลอยู่ในสถานะที่ไม่ consistent ได้



รูปที่ 3-2 แอปพลิเคชันที่ทำงานในระบบแบบกระจาย



รูปที่ 3-3 การใช้งานฐานข้อมูลพร้อม ๆ กัน

■ การที่ผู้ใช้หลายคนสามารถใช้งานข้อมูลเดียวกันพร้อมกันได้

ในระบบแบบกระจายในระดับเอนเตอร์ไพรส์ (enterprise-level distributed system) เป็นรูปแบบที่ไคลเอนต์หลาย ๆ ตัวติดต่อกับแอปพลิเคชันเซิร์ฟเวอร์หลาย ๆ ตัว ถ้าแอปพลิเคชันเซิร์ฟเวอร์ทั้งหมดใช้ฐานข้อมูลเดียวกัน (ดังรูปที่ 3-3) เซิร์ฟเวอร์สามารถที่จะแก้ไขข้อมูลตัวเดียวกันที่อยู่ในฐานข้อมูลได้

ถ้ามีไคลเอนต์สองตัวพยายามแก้ไขข้อมูลเดียวกันที่อยู่ในฐานข้อมูลในเวลาเดียวกันแล้ว คำสั่งของไคลเอนต์ทั้งสองอาจทำงานซ้อนกัน และได้ผลลัพธ์สุดท้ายผิดพลาด ซึ่งการมีข้อมูลที่ไม่ถูกต้องในฐานข้อมูล ในระบบทั่วไปถือว่าเป็นสิ่งที่ยอมรับไม่ได้ (unacceptable) และอาจสร้างความเสียหายอย่างมากให้กับองค์กรดังนั้นจึงจำเป็นต้องหาวิธีการที่จะจัดการกับการแก้ไขข้อมูลพร้อมกันของไคลเอนต์หลาย ๆ ตัว โดยจะต้องไม่ทำให้ผลลัพธ์สุดท้ายที่ได้เป็นผลลัพธ์ที่ไม่ถูกต้อง

3.2.2 ประโยชน์ของทรานแซกชัน

ทรานแซกชัน คือเซตของคำสั่งที่ถูกเอ็กซิกิวต์โดยมองว่าเป็นคำสั่งใหญ่เพียงอันเดียว ทรานแซกชันรับประกันว่าคำสั่งทั้งหมดทำงานสำเร็จ มิฉะนั้นจะต้องไม่มีผลการทำงานจากคำสั่งใดเกิดขึ้นเลย

ทรานแซกชันสามารถแก้ปัญหาที่เกิดจากการทำงานผิดพลาดของระบบเครือข่ายได้ อนุญาตให้ไคลเอนต์หลาย ๆ ตัวใช้งานข้อมูลเดียวกันพร้อมกันได้ และรับประกันว่าข้อมูลทั้งหมดที่คำสั่งเหล่านี้ทำการแก้ไข จะถูกต้องสมบูรณ์ โดยไม่ถูกรบกวนจากการแก้ไขข้อมูลโดยไคลเอนต์อื่น ๆ

การใช้ทรานแซกชันอย่างถูกต้องทำให้สามารถมีหลายไคลเอนต์ทำงานกับฐานข้อมูลได้โดยไม่ขึ้นต่อกัน เช่น ถ้ามีไคลเอนต์สองตัวอ่านและเขียนข้อมูลลงในฐานข้อมูล ไคลเอนต์ทั้งสองจะ mutual exclusive กัน ระบบจัดการฐานข้อมูลจะจัดการเกี่ยวกับการควบคุมการทำงานพร้อมกันที่จำเป็นบนฐานข้อมูล เช่น การล็อกข้อมูล เพื่อไม่ให้งานของแต่ละเซรมีผลกระทบซึ่งกันและกัน

3.2.2.1 ACID Properties

เมื่อใช้งานทรานแซกชันอย่างถูกต้อง คำสั่งจะถูกเอ็กซีคิวต์ โดยมีการรับประกันผลการดำเนินงานทั้งสิ้น 4 อย่างด้วยกัน เรียกว่า ACID properties โดยคำว่า ACID ย่อมาจาก Atomicity , Consistency , Isolation และ Durability

- **Atomicity** - รับประกันว่าคำสั่งทั้งหมดจะถูกรวมเข้าไว้ด้วยกัน และมองเป็นหน่วยของการทำงานเพียงอันเดียว การทำงานของทรานแซกชันจะเป็นไปในลักษณะของ All-or-Nothing คือการอัปเดตข้อมูลในฐานข้อมูลทั้งหมดจะต้องเสร็จสิ้น หรือถ้าเกิดข้อผิดพลาดใด ๆ ขึ้นระหว่างการดำเนินงาน การอัปเดตฐานข้อมูลทั้งหมดนั้นจะต้องไม่เกิดขึ้นเลย โดยการอัปเดตฐานข้อมูลที่เกิดขึ้นก่อนแล้ว จะต้องถูกยกเลิก
- **Consistency** - รับประกันว่าทรานแซกชันจะต้องทำให้ระบบอยู่ในสถานะที่ consistent เมื่อทรานแซกชันทำงานเสร็จเรียบร้อย เช่น การโอนเงินจะต้องไม่มีเงินหายไปจากระบบ แต่ในระหว่างการทำงานของทรานแซกชันนี้ ขณะที่เงินถูกถอนออกมาจากบัญชีแรก ข้อมูลอยู่ในสถานะที่ไม่ consistent แต่เนื่องจากทรานแซกชันจะมองเห็นคำสั่งทั้งหมดเปรียบเสมือนเป็นคำสั่งเดียว ดังนั้นถ้าทรานแซกชันสำเร็จก็คือคำสั่งทั้งหมดจะต้องสำเร็จ แต่ถ้าทรานแซกชันทำงานไม่สำเร็จคำสั่งทั้งหมดก็จะต้องไม่สำเร็จ และการเปลี่ยนแปลงใด ๆ ที่ได้กระทำไประหว่างนั้นจะต้องถูกยกเลิก ตามคุณสมบัติข้อแรกของทรานแซกชัน ดังนั้นข้อมูลจะอยู่ในสถานะที่ consistent เสมอ
- **Isolation** - เป็นการป้องกันทรานแซกชันที่ทำงานพร้อมกัน ไม่ให้เห็นผลลัพธ์ที่ยังไม่สมบูรณ์ที่เกิดจากทรานแซกชันอื่น isolation อนุญาตให้หลายทรานแซกชันสามารถอ่านหรือเขียนฐานข้อมูลโดยไม่รบกวนการมีอยู่ของทรานแซกชันอื่น ถ้าไม่มีคุณสมบัติ isolation สถานะของข้อมูลอาจจะอยู่ในสถานะที่ไม่ consistent ได้ ซึ่งมีประโยชน์มากในกรณีไคลเอนต์หลายตัวแก้ไขฐานข้อมูลในเวลาเดียวกัน โดยแต่ละไคลเอนต์จะทำงานโดยเปรียบเสมือนว่ามีเพียงตัวมันเองเท่านั้นที่กำลังใช้งานฐานข้อมูลอยู่ ทรานแซกชันสร้างคุณสมบัติ isolation โดยการใช้อยู่ synchronization โปรโตคอลต่าง ๆ กับข้อมูลในฐานข้อมูล เช่น ระหว่างการทำงานของทรานแซกชัน ข้อมูลจะถูก

ล็อกและปลดล็อกโดยอัตโนมัติตามความจำเป็น และการอัปเดตข้อมูลของทรานแซกชันอื่น จะไม่มีผลกับข้อมูลที่ทรานแซกชันนี้กำลังทำงานอยู่

- **Durability** - รับประกันว่าการเปลี่ยนแปลงข้อมูลใด ๆ ในฐานข้อมูลที่เสร็จสิ้น (commit) แล้ว จะต้องมีความถาวร คือต้องไม่สูญหายเมื่อเกิดความผิดพลาดใด ๆ ขึ้น เช่น เครื่องคอมพิวเตอร์ แครช ,ระบบเครือข่ายแครช , ฮาร์ดดิสก์แครช หรือไฟฟ้าดับ โดยต้องมีการเก็บ log file ของการอัปเดตข้อมูลใด ๆ ในทรานแซกชัน เมื่อระบบเกิดแครชขึ้น ข้อมูลสามารถสร้างขึ้นใหม่โดยการทำงานตามลำดับใน log file อีกครั้งหนึ่ง

3.2.3 ทรานแซกชันโมเดล

ทรานแซกชันโมเดลเป็นวิธีการที่ต่างกันในการทำทรานแซกชัน มีหลายโมเดลที่ใช้จัดการทรานแซกชัน แต่ละโมเดลจะมีความซับซ้อนและฟีเจอร์ที่ต่างกัน 2 โมเดลที่ได้รับความนิยมมากที่สุดคือ flat transaction และ nested transaction

■ Flat Transactions

flat transaction เป็นทรานแซกชันโมเดลที่เข้าใจได้ง่าย flat transaction เป็นกลุ่มของ operation ที่ทำงานเป็นหน่วยเดียว ถ้า operation ไม่สำเร็จทั้งหมดจะเกิด rollback ขึ้น แต่ถ้าสำเร็จจะเกิด commit กระบวนการของ flat transaction ดังรูปที่ 3-4

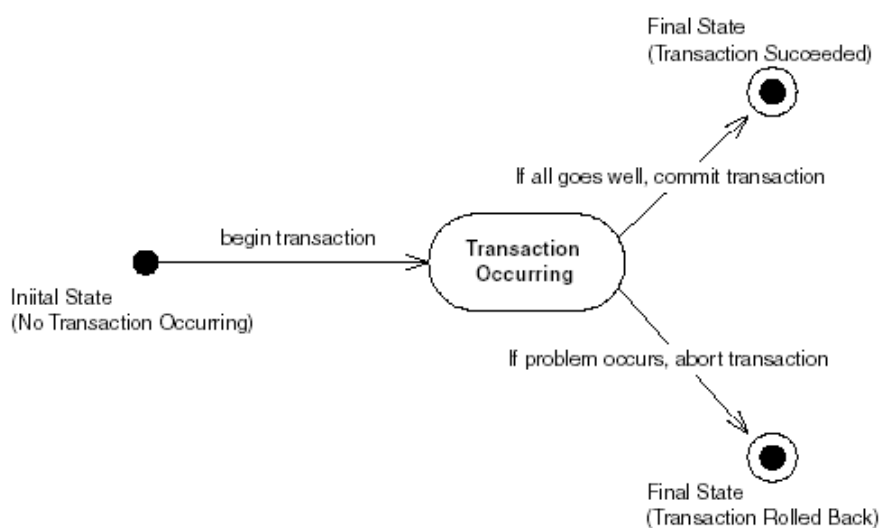
■ Nested Transactions

ตัวอย่างของงานที่ต้องใช้ nested transaction คือปัญหา trip-planning โดยแอปพลิเคชันจะมี operation ดังนี้

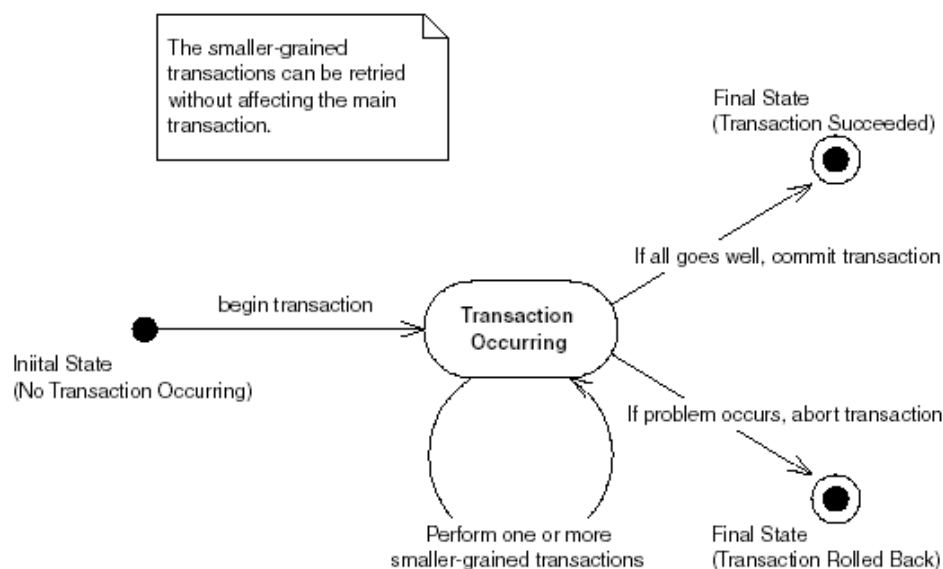
1. ซื้อตั๋วรถไฟจากบอสตันไปยังนิวยอร์ก
2. ซื้อตั๋วเครื่องบินจากนิวยอร์กไปยังลอนดอน
3. ซื้อตั๋วบอลลูนจากลอนดอนไปยังปารีส
4. แอปพลิเคชันพบว่าไม่มีไฟลท์ไปยังปารีส

ถ้าลำดับของ operation นี้ทำกับ flat transaction จะเกิด rollback เท่านั้น เนื่องจากไม่มีไฟลท์ไปยังปารีส แอปพลิเคชันจะต้องไม่เก็บการจองใด ๆ ไว้เลย แต่ตรงจุดนี้อาจมีทางเลือกในการเปลี่ยนการเดินทางด้วยบอลลูนไปเป็นทางอื่น ทำให้เก็บข้อมูลการจองไว้ได้

nested transaction ช่วยในการแก้ปัญหานี้ มันจะยอมให้เกิดทรานแซกชันซ้อนกันได้ รูปที่ 3-5 แสดงแนวคิดของ nested transaction



รูปที่ 3-4 Flat Transaction



รูปที่ 3-5 Nested Transaction

▪ โมเดลอื่น ๆ

ทรานแซกชันโมเดลยังมี chained transaction และ sagas แต่จะไม่อธิบายถึง เพราะ EJB specification ไม่รองรับโมเดลเหล่านั้น รวมถึง nested transaction ด้วย มันจะรองรับเฉพาะ flat transaction เท่านั้น

3.2.4 การใช้งานทรานแซกชันกับ EJB

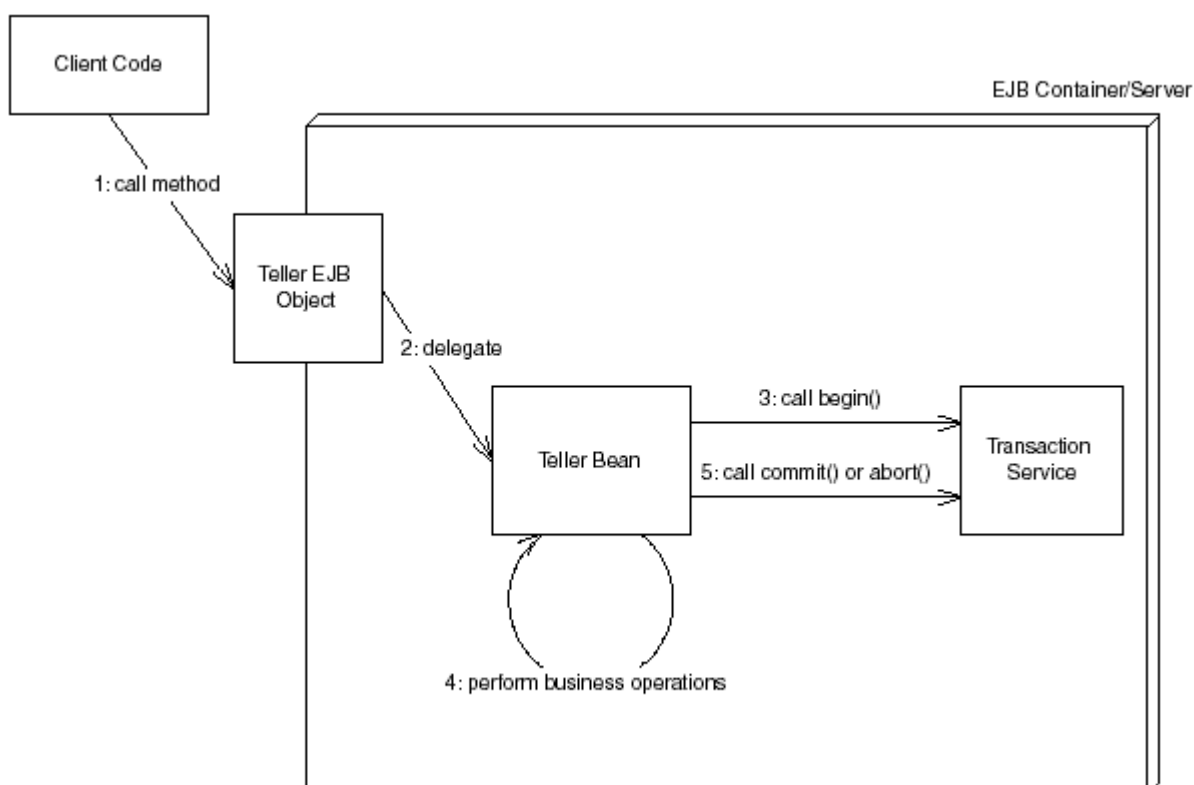
3.2.4.1 วิธีการกำหนดการจัดการทรานแซกชัน

ทรานแซกชันใด ๆ ผลการทำงานของทรานแซกชันจะจบลงด้วยการ commit หรือ abort ใด ๆ อย่างหนึ่งเท่านั้น โดยวิธีการกำหนดขอบเขตของทรานแซกชันสำหรับ EJB มี 2 วิธี

1) Programmatic Transaction

ผู้พัฒนาแอปพลิเคชันเป็นผู้เขียนโปรแกรมลอจิกเกี่ยวกับการควบคุมทรานแซกชันลงไปในโค้ดของแอปพลิเคชันเอง ต้องเป็นผู้ตั้งค่าสั่ง begin , commit หรือ abort เอง

การทำงานของ programmatic transaction แสดงดังรูปที่ 3-6

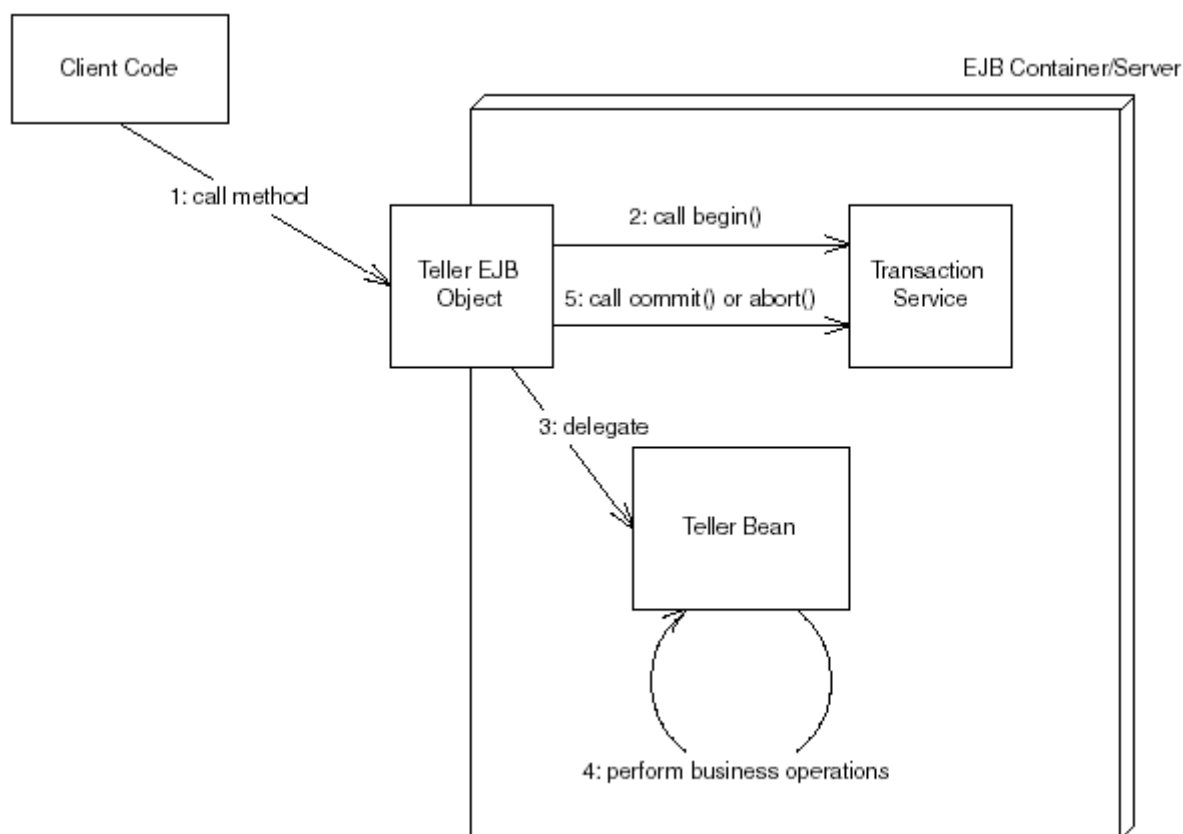


รูปที่ 3-6 bean ที่ทำงานกับ Programmatic Transaction

2) Declarative Transaction

declarative transaction อนุญาตให้คอมโพเนนต์ทำงานในรูปแบบทรานแซกชันโดยอัตโนมัติ enterprise bean ไม่จำเป็นต้องสั่ง begin , commit หรือ abort เอง คอนเทนเนอร์จัดการให้อัตโนมัติ EJB คอนเทนเนอร์จะดักจับการร้องขอที่มีเข้ามาและเริ่มต้นทรานแซกชันอัตโนมัติโดยการสั่ง begin หลังจากนั้นคอนเทนเนอร์ก็จะส่งต่อการร้องขอไปยัง enterprise bean ซึ่งจะทำงานอยู่ในทรานแซกชันที่เกิดขึ้น ถ้าหากมีข้อผิดพลาดใด ๆ bean จะส่งไปบอกคอนเทนเนอร์ให้ยกเลิกทรานแซกชัน เมื่อการทำงานเสร็จสิ้น bean จะส่งการควบคุมกลับมาให้คอนเทนเนอร์ คอนเทนเนอร์จะสั่ง commit หรือ abort โดยขึ้นอยู่กับ

กับว่ามีข้อผิดพลาดเกิดขึ้นระหว่างการทำงานหรือไม่ การทำงานของ declarative transaction แสดงในรูปที่ 3-7



รูปที่ 3-7 bean ที่ทำงานใน Declarative Transaction

declarative transaction ช่วยให้ bean ไม่จำเป็นต้องติดต่อกับ API ของทรานแซกชันโดยตรง โดยที่โค้ดของ bean และไคลเอนต์จะไม่รู้เลยว่ามันทำงานอยู่ภายใต้ทรานแซกชัน

การระบุว่าการทำงานในแบบ declarative หรือ programmatic transaction นั้น EJB กำหนดให้ระบุไว้ในแท็ก persistence-type ที่อยู่ใน ejb-jar.xml ถ้ามีค่าเท่ากับ Bean จะเป็นการระบุให้ bean ควบคุมเอง แต่ถ้าเป็น Container จะให้คอนเทนเนอร์เป็นผู้ควบคุมจะมีค่าดังนี้

```
<persistence-type>Bean</persistence-type>
```

```
<persistence-type>Container</persistence-type>
```

3.2.4.2 การจัดการทรานแซกชันของคอนเทนเนอร์

transaction attribute เป็นค่าที่ระบุว่าคอนเทนเนอร์จะควบคุมทรานแซกชันของ bean อย่างไร โดยสามารถระบุ transaction attribute ที่แตกต่างกันสำหรับแต่ละ bean ในระบบได้โดยไม่ต้องคำนึงถึงว่าจะมี bean ทั้งหมดกี่ตัวที่ทำงานอยู่ร่วมกัน

transaction attribute เป็นส่วนหนึ่งของ deployment descriptor ของแต่ละ bean deployment descriptor ที่มากับ bean จะระบุคุณสมบัติต่าง ๆ ในการทำงานของ bean คอนเทนเนอร์จะจัดการกับทรานแซกชันของ bean ตามที่ได้ดูจาก transaction attribute ใน deployment descriptor และสามารถกำหนด transaction attribute สำหรับทุก ๆ เมธอดของ bean หรือจะระบุแตกต่างกันไปในแต่ละเมธอดก็ได้ โดยถ้ามีการระบุค่าทั้งสองแบบ คอนเทนเนอร์จะยึดถือค่าที่ระบุให้แต่ละเมธอดเป็นหลัก ตัวอย่างของการกำหนด transaction attribute ใน ejb-jar.xml ดังนี้

```
<assembly-descriptor>
  <container-transaction>
    <method>
      <ejb-name>Product</ejb-name>
      <method-name>*</method-name>
    </method>
    <!--
      Transaction attribute. Can be "NotSupported", "Supports", "Required",
      "RequiresNew", "Mandatory", or "Never"
    -->
    <trans-attribute>Required</trans-attribute>
  </container-transaction>
</assembly-descriptor>
```

ค่า transaction attribute ที่เป็นไปได้ มีดังนี้

■ Required

จะใช้งานโหมด Required เมื่อต้องการให้ bean ทำงานในทรานแซกชันเสมอ ถ้ามีทรานแซกชันกำลังทำงานอยู่แล้ว bean จะทำงานร่วมกับทรานแซกชันเดิม แต่ถ้าไม่มีทรานแซกชันกำลังทำงานอยู่ bean จะเริ่มต้นการทำงานในทรานแซกชันใหม่

■ RequiresNew

จะใช้ค่านี้เมื่อต้องการให้มีการเริ่มต้นทรานแซกชันใหม่ทุกครั้งเมื่อ bean ถูกเรียกใช้งาน ถ้ามีทรานแซกชันเดิมกำลังทำงานอยู่ก่อนแล้ว ทรานแซกชันนั้นจะถูกหยุดไว้ชั่วคราว คอนเทนเนอร์จะเริ่มต้นการทำงานของทรานแซกชันใหม่และส่งต่อการร้องขอมายัง bean ใหม่ที่ถูกเรียกใช้งานนี้ เมื่อ bean ทำงานเสร็จสิ้น คอนเทนเนอร์ก็จะสั่ง commit หรือ abort ทรานแซกชันนี้ และกลับไปทำงานต่อในทรานแซกชันเก่า

■ Supports

เมื่อ bean ถูกเรียกใช้งานด้วย Supports มันจะทำงานในทรานแซกชันเมื่อไคลเอนต์นั้นทำงานอยู่ในทรานแซกชันก่อนแล้ว โดยทำงานในทรานแซกชันเดิม แต่ถ้าไคลเอนต์ไม่ได้ทำงานในทรานแซกชัน bean ก็จะไม่เริ่มต้นการทำงานในทรานแซกชันใหม่

■ Mandatory

ระบุว่ามีทรานแซกชันกำลังทำงานอยู่แล้วเมื่อเมธอดของ bean ถูกเรียกใช้งาน ถ้าหากไม่มีทรานแซกชันทำงานอยู่ก่อน *exception javax.ejb.TransactionRequired* จะถูกส่งกลับไปยังผู้เรียก

■ NotSupported

bean จะไม่สามารถทำงานในทรานแซกชันได้ ยกตัวอย่างเช่น สมมติว่าเรามี enterprise bean A และ B เมื่อ A เริ่มต้นทรานแซกชัน และเรียกใช้งาน B หาก B มีการตั้งค่า transaction attribute เป็น NotSupported แล้ว ทรานแซกชันที่ A เริ่มต้นไว้จะหยุดชั่วคราว คำสั่งของ B ทั้งหมดจะทำงานโดยไม่อยู่ในทรานแซกชัน เมื่อ B ทำงานเสร็จเรียบร้อย ทรานแซกชันของ A จะกลับมาทำงานต่อ

■ Never

ใช้ระบุว่า bean จะต้องไม่มีทรานแซกชัน ถ้าไคลเอนต์ที่เรียก bean อยู่ในทรานแซกชันจะเกิด exception ขึ้นมาให้ไคลเอนต์

ตารางที่ 3-1 แสดงผลลัพธ์ที่เกิดจาก transaction attribute

3.2.4.3 การจัดการทรานแซกชันของ bean

การระบุการทำงานทรานแซกชันเป็นแบบ bean ควบคุมทรานแซกชันของ bean เองนั้น EJB กำหนดให้ระบุไว้ในแท็ก *persistence-type* ที่อยู่ใน *ejb-jar.xml* หรือกำหนดในตอนเริ่มสร้าง bean มีค่าเท่ากับ Bean จะเป็นการระบุให้ bean ควบคุมเอง ทำให้ผู้พัฒนาแอปพลิเคชันสามารถเขียนโปรแกรมลอจิกเกี่ยวกับการควบคุมทรานแซกชันลงไปในโค้ดของแอปพลิเคชันเอง และต้องเป็นผู้สั่งคำสั่ง *begin* , *commit* หรือ *rollback* เอง

3.2.5 Transactional Isolation

คุณสมบัติ isolation รับประกันว่า ผู้ใช้ที่ทำงานอยู่พร้อม ๆ กันจะไม่รู้สึกและได้รับผลกระทบจากผู้ใช้อื่น ๆ แม้ว่าผู้ใช้ทั้งหมดนั้นกำลังทำงานกับข้อมูลเดียวกันในฐานข้อมูลก็ตาม โดยสามารถระบุระดับของ isolation ที่ต้องการได้

TRANSACTION ATTRIBUTE	CLIENT'S TRANSACTION	BEAN'S TRANSACTION
Required	none	T2
	T1	T1
	none	T2

RequiresNew	none	T2
RepeatsNew	none	none
	T1	T1
Mandatory	none	error
	T1	T1
NotSupported	none	none
	T1	none
	none	none
	T1	error

ตารางที่ 3-1 ผลลัพธ์ที่เกิดจาก *Transaction Attribute*

3.2.5.1 ระดับของ Isolation

การระบุระดับของ isolation สามารถควบคุมได้ว่าจะให้แต่ละทรานแซกชัน isolate จากกันมากน้อยเพียงใด ถ้าตั้งระดับของ isolation ที่สูง ก็มั่นใจได้ว่าแต่ละทรานแซกชันนั้น จะถูก isolate ออกจากทรานแซกชันอื่น ๆ ทั้งหมด แต่การตั้งระดับของ isolation สูงก็ทำให้ประสิทธิภาพในการทำงานที่ได้ลดลง เนื่องจากในการทำ isolation นั้นใช้วิธีการล็อกค่าในฐานข้อมูลเป็นหลัก แต่ถ้าตั้งระดับของ isolation ไว้ต่ำ ทรานแซกชันก็จะไม่ isolate แต่ประสิทธิภาพในการทำงานร่วมกันของหลาย ๆ ทรานแซกชันจะดีขึ้น

ดังนั้น ต้องพิจารณาให้รอบคอบว่า ต้องการความเป็น isolation สูงต่ำเพียงใดในการทำงาน การกำหนด transaction isolation มีอยู่ทั้งหมด 4 ระดับด้วยกัน ดังนี้

■ READ UNCOMMITTED

เป็นระดับการทำงานที่ต่ำที่สุด ไม่มีการรับประกันคุณสมบัติ isolation เลย แต่จะให้ประสิทธิภาพในการทำงานสูงสุด การทำงานในระดับนี้อาจเกิดปัญหา dirty read ขึ้นได้ โดยหากทรานแซกชันหนึ่งทำงานพร้อมกันไปกับทรานแซกชันอื่น แล้วอีกทรานแซกชันนั้นเขียนข้อมูลโดยยังไม่ได้ commit ทรานแซกชันเดิมจะอ่านค่านั้นเข้ามาได้ทำให้ได้ค่าที่ผิดพลาดไป โดยไม่ขึ้นกับระดับของ isolation ที่อีกทรานแซกชันหนึ่งใช้งานอยู่ นอกจากนี้ยังเกิดปัญหา unrepeatable read และ phantom phenomenon ได้อีกด้วย

■ READ COMMITTED

การทำงานในระดับนี้คล้ายกับระดับ READ UNCOMMITTED แตกต่างกันในระดับนี้โค้ดจะอ่านค่าข้อมูลที่ commit แล้วเท่านั้น จะไม่อ่านค่าที่ถูกเขียนลงไปยังฐานข้อมูลแล้วแต่ยังไม่ commit ดังนั้นระดับของ isolation ระดับนี้จึงสามารถแก้ปัญหา dirty read ได้ แต่ยังไม่สามารถแก้ปัญหาที่ซับซ้อน เช่น ปัญหา unrepeatable read และ phantom phenomenon

■ REPEATABLE READ

เป็นการรับประกันที่สูงขึ้นไปจาก READ COMMITTED เมื่อใดก็ตามที่อ่านค่าข้อมูลที่ commit แล้วจากฐานข้อมูล สามารถจะอ่านค่าข้อมูลนั้นซ้ำได้และค่าที่ได้จะเป็นค่าที่เหมือนกับค่าที่เราอ่านขึ้นมาได้ครั้งแรก ซึ่งระดับของ isolation ที่ READ COMMITTED หรือต่ำกว่า ทรานแซกชันอื่นที่ทำงานอยู่พร้อมกันอาจจะ commit ข้อมูลในระหว่างการอ่านข้อมูลแต่ละครั้งได้ ทำให้ค่าของข้อมูลเดียวกันที่อ่านได้ในแต่ละครั้งไม่ตรงกัน การทำงานในโหมดนี้แก้ปัญหาที่เรียกว่า unrepeatable read

■ SERIALIZABLE

สามารถแก้ปัญหา phantom phenomenon คือการที่เซตของข้อมูลใหม่ถูกเพิ่มเข้ามาในฐานข้อมูลระหว่างการอ่านข้อมูลของทรานแซกชัน รวมถึงแก้ปัญหาค่าอื่น ๆ ที่กล่าวมาก่อนหน้านี้ทั้งหมด รับประกันว่าทุก ๆ ทรานแซกชันจะทำงานเป็นลำดับกับทรานแซกชันอื่น ๆ และมีคุณสมบัติ ACID properties ของฐานข้อมูลครบถ้วน หมายความว่าแต่ละทรานแซกชันจะทำงานแยกกับทรานแซกชันอื่น ๆ อย่างแท้จริง ตารางที่ 3-2 จะแสดงถึงปัญหาที่อาจเกิดในแต่ละระดับของ isolation

ISOLATION LEVEL	DIRTY READ?	UNREPEATABLE READS?	PHANTOM READ?
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

ตารางที่ 3-2 ระดับของ Isolation

การกำหนด isolation ใน EJB ในกรณีของ bean-managed persistent จะกำหนด isolation โดยใช้ resource manager API เช่น JDBC โดยการเรียก `java.sql.Connection.SetTransactionIsolation(...)` ส่วนในกรณีของ container-managed persistent จะไม่สามารถกำหนดระดับของ isolation ใน deployment descriptor ได้ การกำหนดระดับของ isolation จะขึ้นอยู่กับผลิตภัณฑ์ที่ใช้เป็นคอนเทนเนอร์หรือฐานข้อมูล

3.2.6 ทรานแซกชันแบบกระจาย

นอกจากทรานแซกชันพื้นฐาน คือแอปพลิเคชันเซิร์ฟเวอร์หนึ่งตัวเชื่อมต่อกับฐานข้อมูลหนึ่งตัวแล้ว ยังสามารถที่จะทำ distributed transaction ได้เช่นเดียวกัน โดยขึ้นอยู่กับวิธีการรองรับของแอปพลิเคชันเซิร์ฟเวอร์ ซึ่ง distributed transaction ยึดหลักการเดียวกับทรานแซกชันธรรมดา โดยที่หากมีคอมพิวเตอร์หนึ่งบนเครื่องคอมพิวเตอร์เครื่องหนึ่ง abort ทรานแซกชัน ทรานแซกชันทั้งหมดจะต้องถูก abort ด้วย

เช่นกัน distributed transaction สามารถทำงานได้กับหลายแอปพลิเคชันเซิร์ฟเวอร์และหลายฐานข้อมูล หมายความว่าคอมพิวเตอร์ที่ตีพลอยในแอปพลิเคชันเซิร์ฟเวอร์หนึ่ง ทำงานในทรานแซกชันเดียวกันร่วมกับอีกคอมพิวเตอร์ซึ่งถูกตีพลอยในอีกแอปพลิเคชันเซิร์ฟเวอร์ ในการทำงานบิซิเนสเมธอดหนึ่ง ๆ เครื่องหลายเครื่องอาจจำเป็นต้องทำงานร่วมกัน distributed transaction จึงอนุญาตให้หลายแอปพลิเคชันเซิร์ฟเวอร์ซึ่งอาจเป็นซอฟต์แวร์จากผู้พัฒนาคนละรายกันทำงานร่วมกันในทรานแซกชันได้

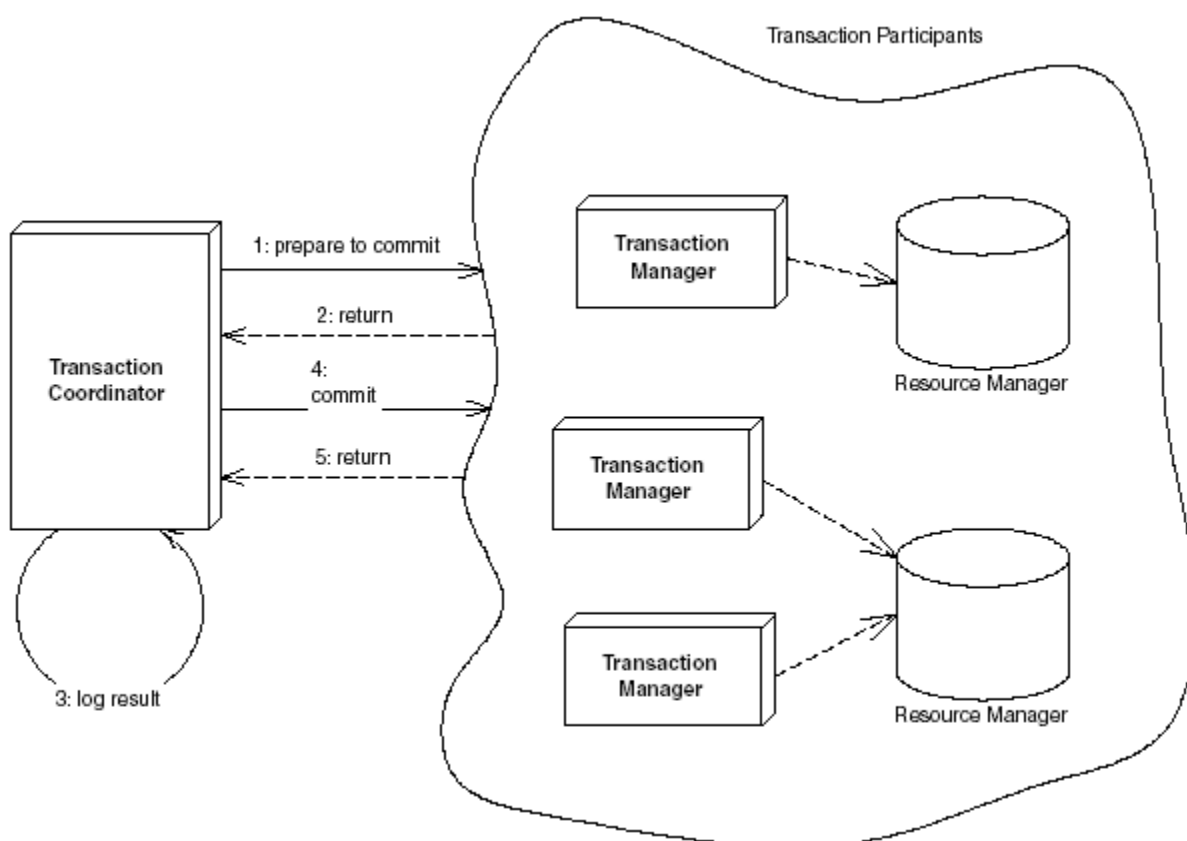
3.2.6.1 Durability และโปรโตคอล Two-Phase Commit

durability รับประกันว่า การอัปเดต resource ที่ commit แล้วทุกครั้งจะต้องมีความถาวร สามารถอิมพลีเมนต์ได้ไม่ยากถ้าทำงานกับฐานข้อมูลเพียงฐานข้อมูลเดียว แต่กรณีที่มีหลาย resource manager เข้ามาเกี่ยวข้องด้วย หากมี resource หนึ่งเกิดความผิดพลาดในการทำงานใด ๆ ขึ้น เช่น ฐานข้อมูลแครชจะต้องมีวิธีที่จะกู้ข้อมูลกลับคืนมาให้ได้

วิธีหนึ่งในการแก้ปัญหานี้คือ การบันทึก log ของคำสั่งที่ทำงานกับฐานข้อมูลทั้งหมดก่อนที่จะคำสั่งนั้นจะทำงานจริง จะช่วยให้สามารถกู้ข้อมูลคืนมาได้ โดยดูข้อมูลได้จาก log file และทำการอัปเดตข้อมูลต่าง ๆ ใหม่อีกครั้ง ด้วยวิธีการนี้ทรานแซกชันจะเสร็จสิ้นในสองเฟส

- เฟสที่หนึ่ง - เริ่มต้นโดยส่งข้อความ before commit ไปยังทุก resource ที่เกี่ยวข้องในทรานแซกชัน ณ จุดนี้ resource ที่เกี่ยวข้องในทรานแซกชันจะมีโอกาสที่จะยกเลิกทรานแซกชันเป็นครั้งสุดท้าย ถ้าหากมี resource ใดตัดสินใจยกเลิกทรานแซกชัน ทรานแซกชันทั้งหมดก็จะถูกยกเลิก และการอัปเดต resource ต่าง ๆ ก็จะไม่เกิดขึ้น มิฉะนั้นทรานแซกชันก็จะทำงานต่อไปและไม่มีการหยุดนอกจากจะมีความผิดพลาดเกิดขึ้น เพื่อป้องกันความผิดพลาด การอัปเดต resource ใด ๆ ในแต่ละครั้งจะถูกบันทึกลงใน log หรือ journal ที่มีความคงทน (persistent) โดยข้อมูลเหล่านี้จะไม่สูญหายไปเมื่อมีการแครชเกิดขึ้น และสามารถใช้อ้างอิงข้อมูลเหล่านี้ในการอัปเดต resource ต่าง ๆ ได้อย่างถูกต้อง
- เฟสที่สอง - จะเกิดขึ้นก็ต่อเมื่อเฟสที่หนึ่งจบลงโดยไม่มีการยกเลิกการทำงาน ณ จุดนี้ทุก resource manager ซึ่งอาจถูกควบคุมจากคนละที่ จะทำการอัปเดตข้อมูลจริง ๆ

การแบ่งจุดที่ทรานแซกชันทำงานสำเร็จออกเป็นสองเฟสนี้ เรียกว่าโปรโตคอล two-phase commit หรือ 2PC มันมีประโยชน์มากเนื่องจากทำให้หลาย transaction manager และ resource manager ทำงานร่วมกันในทรานแซกชันแบบกระจายได้ หากมีใครก็ตามระบุว่าทรานแซกชันควรถูกยกเลิก ทุก resource ก็จะต้องถูก roll back ดังรูปที่ 3-8



รูปที่ 3-8 ทรานแซกชันแบบกระจายที่มีการใช้โปรโตคอล *Two-Phase Commit*

ในระบบ two-phase commit แบบกระจาย มี transaction manager หลักอยู่หนึ่งตัว เรียกว่า distributed transaction coordinator ทำหน้าที่ควบคุมทรานแซกชันทั้งหมด และจัดการกับแต่ละคำสั่งที่มาจากหลาย ๆ transaction manager ในเครือข่าย ขั้นตอนต่อไปนี้เป็นขั้นตอนที่เกิดขึ้นในทรานแซกชัน two-phase commit แบบกระจาย

- 1) transaction coordinator ส่งข้อความ prepare to commit ไปยังทุก transaction manager ที่เกี่ยวข้อง
- 2) แต่ละ transaction manager อาจส่งผ่านข้อความนี้ไปยังแต่ละ resource manager ที่สัมพันธ์กับ transaction manager นั้น
- 3) แต่ละ transaction manager รายงานกลับไปยัง transaction coordinator หากทุก transaction manager ยินยอมที่จะ commit คำสั่ง commit ที่กำลังจะเกิดขึ้นจะถูกบันทึกลง log เพื่อใช้ในกรณีที่เกิดความผิดพลาดขึ้น
- 4) สุดท้าย transaction coordinator จะบอกแต่ละ transaction manager ให้ทำการ commit แต่ละ transaction manager จะเรียก resource manager ให้อัปเดตข้อมูล ถ้าหากมีข้อผิดพลาดใด ๆ เกิดขึ้น log จะถูกใช้ในการทำขั้นตอนนี้อีกครั้ง

3.3 Asynchronous and Synchronous

ถ้า เว็บเซิร์ฟเวอร์ ที่คุณกำลัง ออกแบบ มีกระบวนการที่อาจจะใช้เวลานานในการดำเนินการคุณจะต้องออกแบบส่วนติดต่อผู้ใช้ ของ เว็บเซิร์ฟเวอร์ โดยที่ไคลเอนต์ไม่ต้องหยุดรอกระบวนการ(method)นั้นเสร็จ โดยทำให้เป็นเว็บเซิร์ฟเวอร์ของคุณเป็นอะซิงโครนัส(asynchronous) โดยในการออกแบบคุณสามารถทำได้โดยให้กระบวนการนั้นๆ(ที่ใช้เวลานาน) ส่งค่าผลการรับคำร้องขอกลับไปให้ไคลเอนต์ก่อนในทันที หลังจากนั้นค่อยส่งผลลัพธ์ไปให้ไคลเอนต์เมื่อกระบวนการนั้นทำงานเสร็จแล้ว

มันเป็นเรื่องสำคัญที่จะแยกแยะระหว่าง เว็บเซิร์ฟเวอร์ที่เป็นอะซิงโครนัส กับ กระบวนการที่เป็นอะซิงโครนัส(กระบวนการที่ ส่งค่ากลับทันที และ ส่งค่าเป็น วอย(void)) แต่เว็บเซิร์ฟเวอร์ ที่เป็นอะซิงโครนัสนั้นไม่จำเป็นที่กระบวนการต่างๆของมันต้องเป็น อะซิงโครนัส ยกตัวอย่าง เช่นเว็บเซิร์ฟเวอร์ที่ใช้กระบวนการเป็นอะซิงโครนัส เมื่อได้รับการร้องขอจากไคลเอนต์จะส่งค่า การยอมรับ(acknowledgement) กลับไปให้ไคลเอนต์ แต่ไม่ใช่ผลลัพธ์หลังจากนั้นจะเรียกซิงโครนัส คอลแบ็ก(synchronous callback ของไคลเอนต์โดยส่งค่าผลลัพธ์กลับไปและรอรับการยืนยันจากไคลเอนต์ โดยทั้งกระบวนการของเว็บเซิร์ฟเวอร์ และ คอลแบ็กต่างก็เป็นซิงโครนัสทั้งคู่ แต่เว็บเซิร์ฟเวอร์เป็น อะซิงโครนัส

มี 2 วิธีในการออกแบบ อะซิงโครนัส เว็บเซิร์ฟเวอร์

1. สร้างกระบวนการที่เริ่มต้นการร้องขอ และ คอลแบ็ก เพื่อส่งส่งค่าผลลัพธ์กลับ
2. สร้างกระบวนการที่เริ่มต้นการร้องขอ และ เพิ่มกระบวนการ ที่ส่งค่าผลลัพธ์กลับ สถานะของการร้องขอนั้น(ยังไม่เสร็จ/เสร็จแล้ว) วิธีการนี้เรียกว่า การติดต่อแบบสอบถาม(polling interface)

คุณอาจจะต้อง สร้าง โดยใช้ทั้ง 2 วิธี ใน เว็บเซิร์ฟเวอร์ ของคุณ เนื่องจากคุณสามารถอำนวยความสะดวก ของทั้ง ไคลเอนต์ ที่สามารถจัดการกับ คอลแบ็ก ได้ และ ที่ไม่สามารถจัดการได้ (โดยใช้การติดต่อแบบสอบถาม)

3.3.1 การใช้คอลแบ็กเพื่อที่จะระบุเหตุการณ์กับไคลเอนต์

คอลแบ็ก ใช้ในการเตือน ไคลเอนต์ ของ เว็บเซิร์ฟเวอร์ ว่ามีบางเหตุการณ์เกิดขึ้น เช่น คุณต้องการจะบอกกับ ไคลเอนต์ ว่า ผลลัพธ์ที่ ไคลเอนต์ร้องขอมาเสร็จแล้ว

การเพิ่ม คอลแบ็ก ให้กับส่วนติดต่อของ เว็บเซิร์ฟเวอร์ นั้นง่าย แต่คุณต้องตระหนักว่า ไม่ใช่ว่าทุกไคลเอนต์ จะรองรับการติดต่อแบบคอลแบ็กได้

3.3.2 การใช้การติดต่อแบบสอบถามแทนที่ใช้คอลแบ็ก

ไคลเอ็นต์ บางรายของ เว็บเซอร์วิส อาจไม่สามารถจัดการกับ คอลแบ็ก ได้ คุณสามารถทำให้ ไคลเอ็นต์ ที่ไม่สามารถรับ คอลแบ็ก ใช้ เว็บเซอร์วิสของคุณได้โดยการเพิ่มการติดต่อแบบสอบถามให้เป็นอีกทางเลือกหนึ่งของ ไคลเอ็นต์ โดยในการติดต่อแบบสอบถามจะสร้าง กระบวนการ ที่ ไคลเอ็นต์ สามารถใช้เป็นตัวย่อยตรวจสอบได้ว่า ผลลัพธ์ของการร้องขอก่อนหน้านี้ของตนพร้อมแล้วหรือยัง ดัง โปรแกรมตัวอย่างข้างล่าง

```
public class Conversation {

    /**
     * @jws:operation
     * @jws:conversation phase="start"
     */

    public void startRequest()
    {
        ...
    }

    /**
     * @jws:operation
     * @jws:conversation phase="continue"
     */

    public String getRequestStatus()
    {
        ...
    }
}
```

```

/**
 * @jws:operation
 * @jws:conversation phase="finish"
 */
public void terminateRequest()
{
}
}

```

ไคลเอนต์ จะเรียก กระบวนการ startRequest เพื่อเริ่มการร้องขอหลังจากนั้น ไคลเอนต์ สามารถเรียก getRequestStatus เป็นระยะๆ เพื่อตรวจสอบผลลัพธ์ ระหว่างการเรียก getRequestStatus ไคลเอนต์ จะสามารถทำกระบวนการอื่นๆ ได้ โดย getRequestStatus จะส่งค่ากลับว่าการร้องขอนั้นกำลังทำงานอยู่ จนกว่าการร้องขอจะเสร็จโดยครั้งถัดไปถ้า ไคลเอนต์ เรียก getRequestStatus หลังจากที่มีการร้องขอเสร็จ แล้วผลลัพธ์ จะถูกส่งกลับไปที่ ไคลเอนต์ โดย ไคลเอนต์ สามารถหยุดการติดต่อได้โดยใช้ กระบวนการ terminateRequest